TEMPLATE MILESTONE 2

ARSITEKTUR, MODEL DATA, KONTRAK API, DAN KEAMANAN

CIE516 — Perancangan Aplikasi Mobile · Semester Ganjil 2026/2027

Dosen Pengampu: Ary Prabowo, S.Komp., M.Kom.

Isi seluruh bagian pada template ini, lalu unggah sebagai PDF ke LMS dan simpan salinannya di folder /docs repositori kelompok. Batas serah: Sesi 5, pukul 23.59. Dokumen ini menjadi acuan kerja sampai akhir semester — apa yang Anda tulis di sini akan ditagih saat aplikasi dinilai.

Nama kelompok	Kelompok 2 — KH002 (mengacu pada nama repositori; konfirmasi identitas kelas)
Judul aplikasi	LangkahAwal — Aplikasi Pemantauan Tumbuh Kembang Anak Usia 0–5 Tahun
Anggota dan peran	Sakha — [nama lengkap/NIM dan peran perlu dikonfirmasi] Yusuf — [nama lengkap/NIM dan peran perlu dikonfirmasi] Pembagian Project Manager, UI/UX, Flutter, Firebase, dan pengujian belum diberikan.

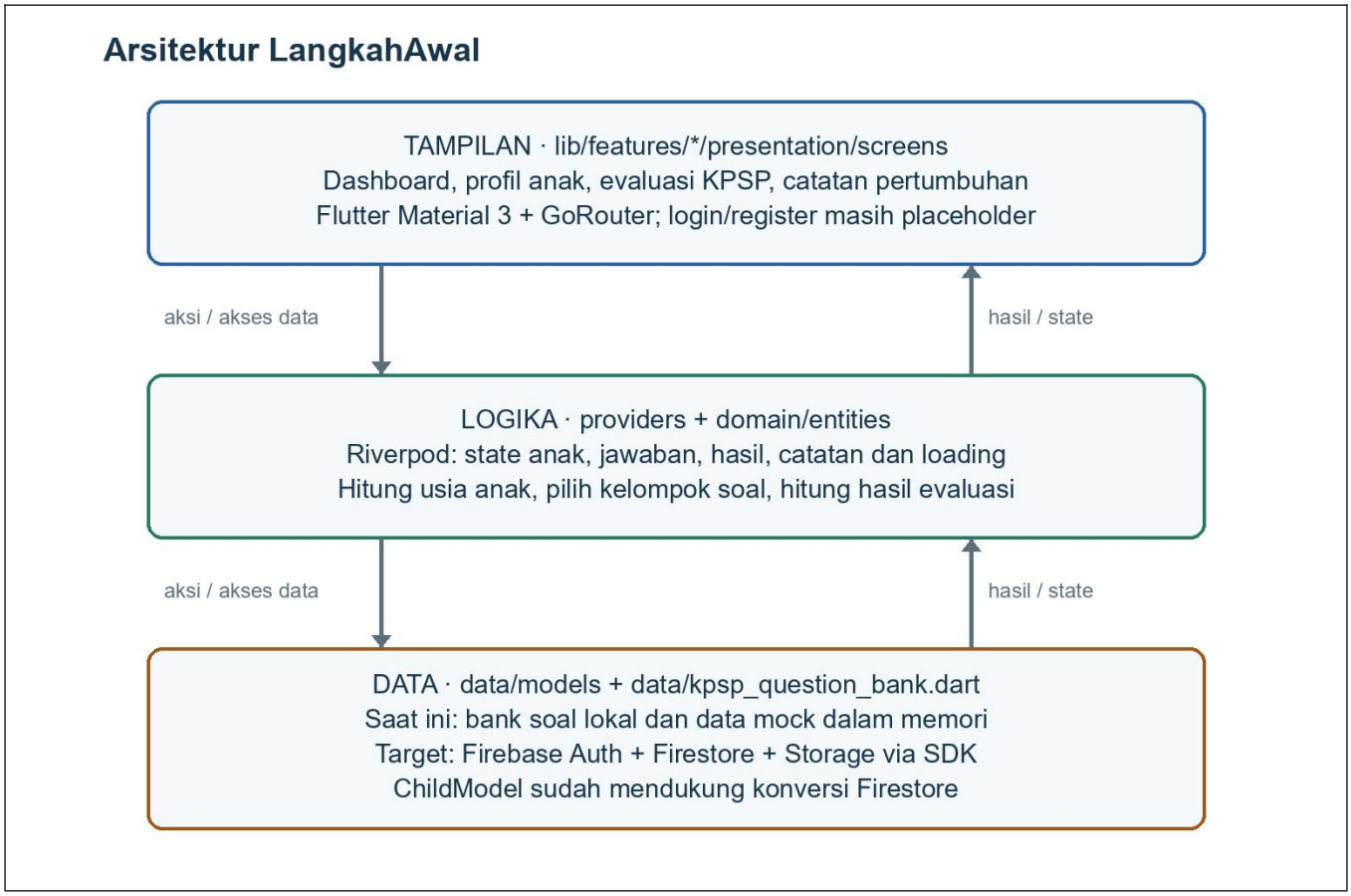
Status dokumen: DRAF TEKNIS, disusun dari kode proyek pada 13 Oktober 2026. Bagian bertanda “saat ini” menggambarkan implementasi yang tersedia. Bagian “target” adalah rancangan, belum berarti fitur selesai. Identitas/peran anggota dan nomor user story perlu dicocokkan dengan dokumen Milestone 1 sebelum dikumpulkan.

Istilah yang dipakai di dokumen ini

Lapisan	Pembagian isi aplikasi menjadi tiga tumpukan tugas: tampilan (yang dilihat dan disentuh pengguna), logika (aturan dan perhitungan), dan data (tempat data diambil dan disimpan). Dipisah supaya mengubah satu bagian tidak merusak bagian lain.
Endpoint	Satu alamat di server yang dipanggil aplikasi untuk satu keperluan tertentu, misalnya mengambil daftar kunjungan. Seperti nomor loket di kantor — tiap loket melayani satu urusan.
Kontrak API	Kesepakatan tertulis tentang apa yang dikirim ke sebuah endpoint dan apa yang dikembalikannya, termasuk kalau terjadi kesalahan.
Entitas	Satu jenis benda yang datanya disimpan aplikasi, misalnya Ibu, Kunjungan, atau Pengingat. Satu entitas biasanya menjadi satu tabel di basis data.
Mitigasi	Langkah nyata untuk mengurangi sebuah risiko.

Bagian 1 · Diagram Arsitektur Berlapis

Gambarkan tiga lapisan aplikasi Anda — tampilan, logika, dan data — beserta hubungan antar lapisannya. Boleh dibuat dengan alat apa pun, lalu disisipkan sebagai gambar.



LAPISAN	APA YANG ADA DI DALAMNYA
Tampilan	Flutter Material 3: ChildListScreen (dashboard), AddChildScreen, MilestoneScreen, TrackingScreen. LoginScreen dan RegisterScreen masih placeholder. GoRouter mengatur navigasi; widget membaca state dengan ref.watch dan mengirim aksi dengan ref.read. Lokasi: lib/features/*/presentation/screens dan lib/core/routing.
Logika	Riverpod mengelola daftar anak, jawaban, hasil, catatan dan loading. ChildEntity menghitung ageInMonths/ageFormatted; KpspQuestionBank.getNearestAge() memilih kelompok umur; MilestoneResultEntity.calculateStatus() menghitung hasil. Validasi form dan sebagian koordinasi simpan masih berada di screen. Target: operasi Firebase dipisahkan dari widget, bukan memanggil SDK langsung dari tampilan.
Data	Saat ini: bank soal statis lokal dan state mock dalam memori. ChildModel menyediakan konversi Firestore; GrowthRecordModel menyediakan JSON tanggal ISO 8601, belum konversi Timestamp. AuthRepository dan GrowthRecordRepository baru kontrak abstrak, belum implementasi. Target: Firebase Auth, Cloud Firestore, Firebase Storage. SecureStorageService tersedia untuk user_id/user_email tetapi belum terhubung ke sesi login.

Hubungan: tampilan → aksi provider/logika → sumber data; data/state yang berubah → provider → tampilan dibangun ulang. Diagram memisahkan tanggung jawab, bukan klaim bahwa seluruh Clean Architecture sudah selesai. main.dart belum menginisialisasi Firebase dan router sementara dimulai dari /home.

Bagian 2 · Cara Mengelola Data Sementara di Aplikasi

Pilih satu pola, lalu jelaskan alasannya. Setelah ditetapkan di sini, pola itu dipakai konsisten sampai akhir semester.

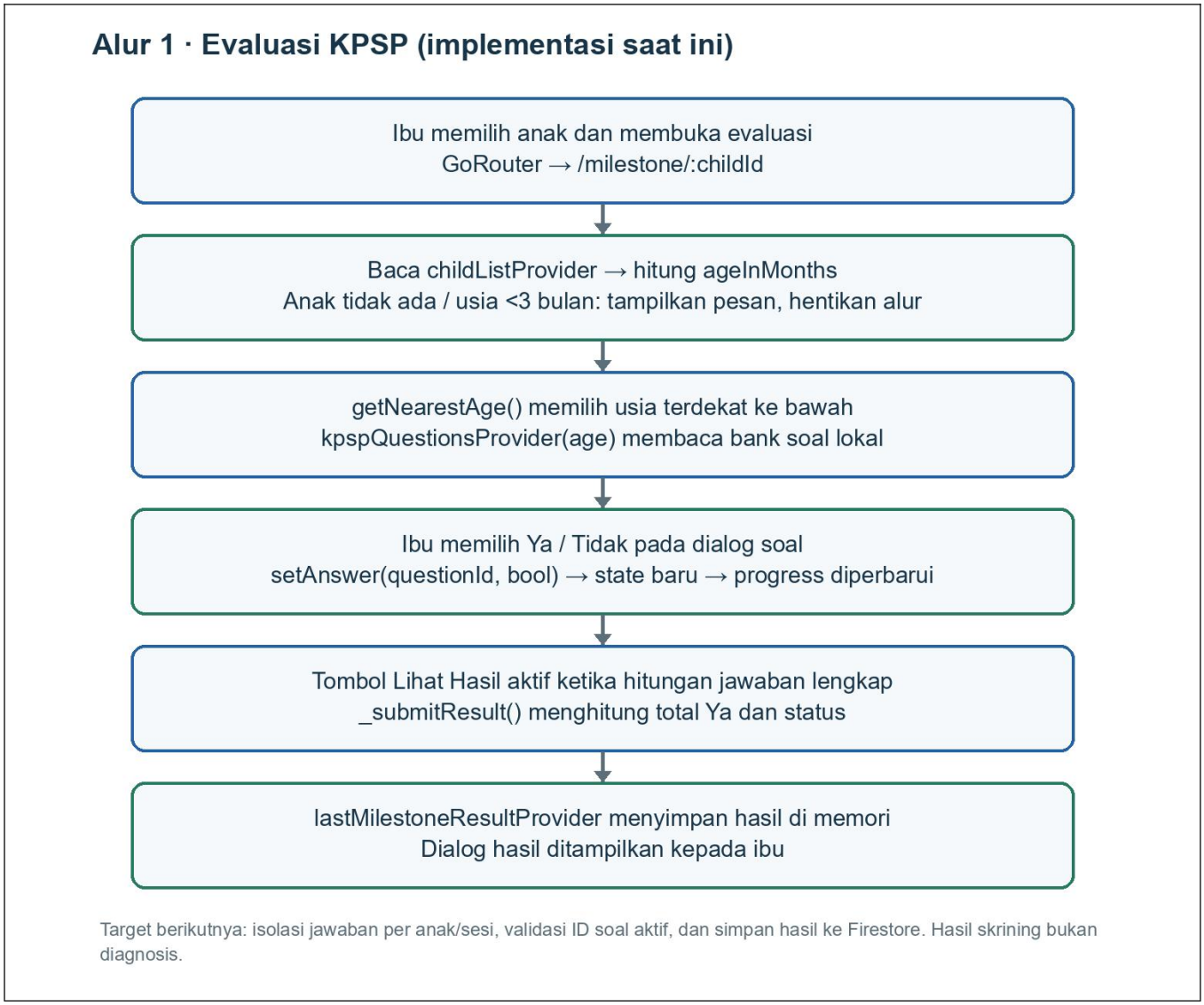
Pola yang dipilih	Riverpod: StateProvider untuk state sederhana, StateNotifierProvider untuk kumpulan data yang berubah, dan Provider.family untuk soal berdasarkan usia.
Alasan memilihnya	Sudah dipakai di proyek; UI reaktif tanpa meneruskan state melalui banyak widget. Provider bisa di-override saat pengujian dan tidak bergantung pada BuildContext. Satu sumber state menghindari salinan data yang tidak sinkron. State memori bukan penyimpanan permanen.
Bagian aplikasi yang memakainya	childListProvider, selectedChildProvider, isLoadingProvider; milestoneAnswersProvider, lastMilestoneResultProvider, kpspQuestionsProvider(ageMonth); growthRecordsProvider dan isSavingRecordProvider. Provider.family yang sudah ada memakai ageMonth, bukan childId. Target: jawaban dipisah per anak/sesi; state sensitif dibersihkan saat logout; loading/error ditampilkan dan input tidak dihapus jika penyimpanan gagal.

Bagian 3 · Alur Perjalanan Data

Gambarkan minimal dua alur utama aplikasi Anda, dari pengguna menekan tombol sampai data kembali tampil di layar. Pilih alur yang paling sering dipakai.

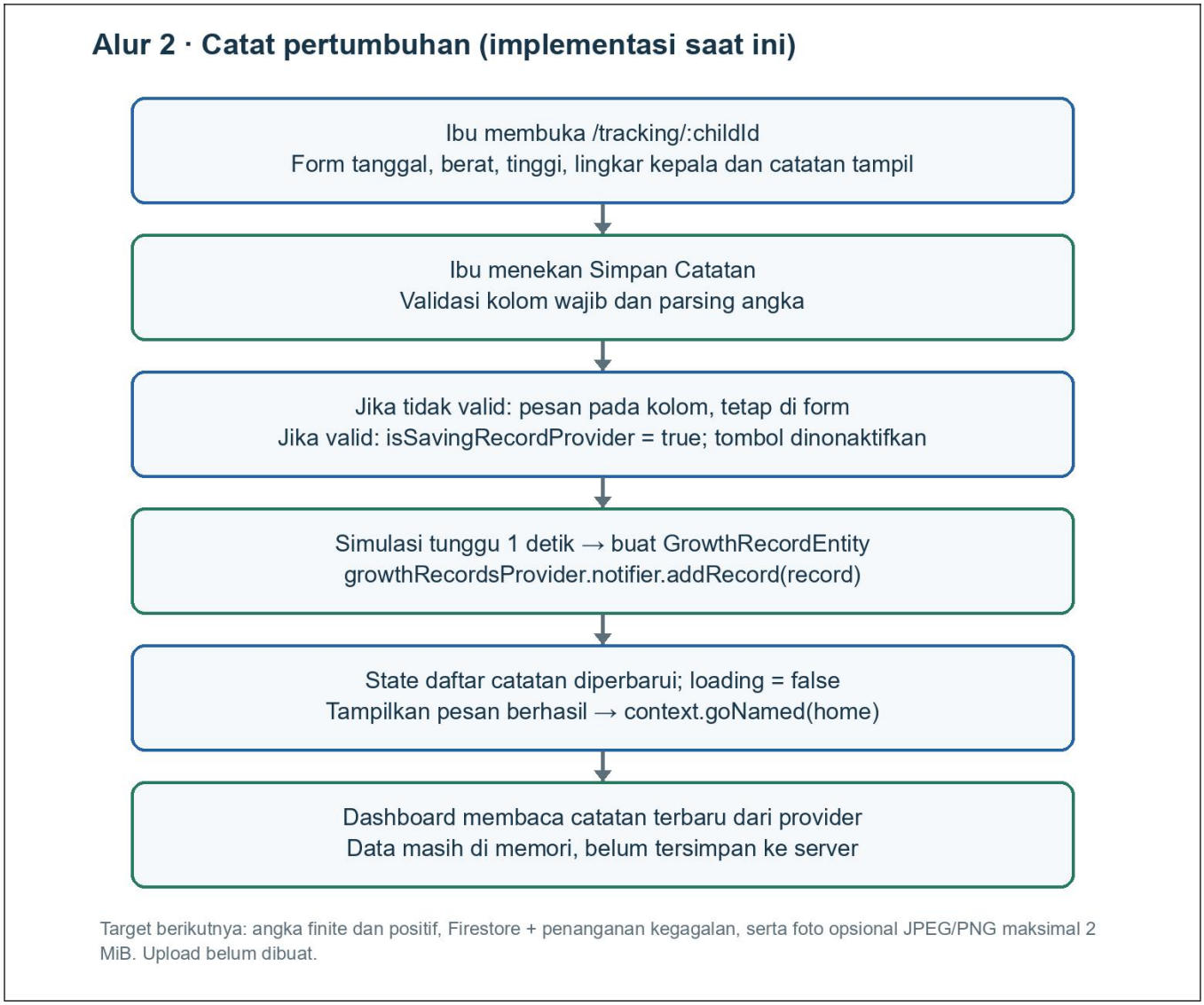
Alur 1 — nama alur: Evaluasi KPSP

Implementasi saat ini memilih kelompok usia terdekat ke bawah dari [3,6,9,12,15,18,21,24,30,36,42,48,54,60]. Usia <3 bulan belum mendapat soal. Kode saat ini memakai rasio Ya/jumlah soal: ≥0,9 Sesuai, ≥0,7 Meragukan, sisanya Penyimpangan. Ini deskripsi kode, bukan validasi medis. Bank soal, gambar pendukung, aturan usia dan penilaian harus diverifikasi terhadap formulir KPSP otoritatif sebelum digunakan secara klinis.



Celah yang harus ditutup: state jawaban masih global; jumlah jawaban saja belum menjamin semua ID soal aktif terjawab. Target simpan: periksa himpunan ID jawaban tepat sama dengan soal aktif, hitung ulang total/status, lalu simpan hasil di childId yang benar. Pada kondisi offline/gagal, pertahankan jawaban dan tampilkan status belum tersinkron, bukan mengklaim berhasil di server.

Alur 2 — nama alur: Mencatat pertumbuhan anak



Validasi saat ini baru wajib isi dan double.tryParse; angka negatif/non-finite belum ditolak. Target: angka harus finite dan >0, tanggal tidak setelah hari ini atau sebelum lahir, dan pemilik anak diperiksa. Foto belum memiliki picker/upload. Setelah simpan mock (delay 1 detik), kode menampilkan SnackBar lalu navigasi ke home; belum ada riwayat cloud.

Bagian 4 · Model Data

Daftarkan seluruh entitas beserta atribut dan tipenya. Kolom terakhir wajib diisi: setiap entitas harus bisa ditunjuk ke minimal satu user story dari dokumen analisis kebutuhan. Entitas yang tidak terpakai satu user story pun dihapus.

Referensi user story berikut adalah pemetaan sementara dari kebutuhan percakapan; dokumen analisis kebutuhan/Milestone 1 tidak tersedia di workspace. Jangan menganggap label US-A sampai US-F sebagai nomor resmi. Cocokkan label/kalimat ini dengan dokumen tersebut sebelum penyerahan.
US-A (sementara): Sebagai ibu, saya ingin membuat akun, masuk dan keluar agar data anak hanya dapat saya akses.
US-B (sementara): Sebagai ibu, saya ingin menambah, melihat, mengubah dan menghapus profil anak agar pemantauan sesuai anak yang dipilih.
US-C (sementara): Sebagai ibu, saya ingin mengisi KPSP sesuai usia dan melihat hasil/riwayat agar dapat memantau perkembangan serta berkonsultasi bila diperlukan.
US-D (sementara): Sebagai ibu, saya ingin mencatat dan melihat berat, tinggi, lingkar kepala, catatan serta foto opsional agar pertumbuhan terdokumentasi.
US-E (sementara): Sebagai ibu, saya ingin pengingat lokal agar tidak melewatkan pencatatan/evaluasi.
US-F (sementara): Sebagai ibu, saya ingin menjadwalkan penghapusan akun dengan masa pembatalan 30 hari agar dapat mengendalikan data pribadi.

ENTITAS	ATRIBUT DAN TIPENYA	DIPAKAI OLEH USER STORY
UserEntity — tersedia; persistensi target	id: String (Firebase uid); email: String; name: String?; createdAt: DateTime. Ekstensi target US-F, belum ada di entity: deletedAt: DateTime?. purgeAfter: DateTime? dihitung dari deletedAt + 30 hari, tidak disimpan. Password/token tidak disimpan dalam dokumen user.	US-A; US-F untuk ekstensi penghapusan akun.
ChildEntity — tersedia	id: String; parentId: String; name: String; birthDate: DateTime; gender: Gender {male, female}; childOrder: int (default 1); photoUrl: String?; createdAt: DateTime. Turunan, tidak disimpan: ageInMonths: int; ageFormatted: String.	US-B; digunakan pula US-C dan US-D sebagai identitas anak.
MilestoneQuestionEntity — bank lokal	id: String; ageMonth: int; aspect: MilestoneAspect {grossMotor, fineMotor, speech, social}; question: String; stimulationTip: String?; isRedFlag: bool (default false). Bukan koleksi per pengguna; saat ini 139 soal: 42 bulan berisi 9, kelompok lain 10. Jumlah/isi tersebut masih perlu validasi sumber.	US-C — memilih dan menjawab soal sesuai kelompok usia.
MilestoneResultEntity — tersedia	id: String; childId: String; ageMonthEvaluated: int; answers: Map<String,bool>; totalYes: int; status: MilestoneStatus {sesuai, meragukan, penyimpangan}; evaluatedAt: DateTime. Target persistensi menambahkan questionBankVersion: String agar hasil historis tidak tercampur setelah bank soal berubah.	US-C — hasil dan riwayat evaluasi per anak.
GrowthRecordEntity — tersedia	id: String; childId: String; recordDate: DateTime; weightKg: double; heightCm: double; headCircumferenceCm: double?; note: String?; photoUrl: String?; createdAt: DateTime.	US-D — catatan antropometri, catatan teks, foto opsional.

ENTITAS	ATRIBUT DAN TIPENYA	DIPAKAI OLEH USER STORY
PengingatLokal — rancangan, belum ada entity/fitur	id: int (ID notifikasi); childId: String; type: String {kpsp, growth}; scheduledAt: DateTime; enabled: bool. Disimpan pada perangkat; tidak membutuhkan collection/API cloud dalam MVP. Maksimal satu pengingat aktif per anak dan jenis.	US-E — jadwal pengingat pencatatan atau evaluasi.

Hubungan antar entitas

• Satu User mempunyai banyak Child; setiap Child hanya memiliki satu parentId, tanpa berbagi akun/pengasuh pada MVP.
• Satu Child mempunyai banyak MilestoneResult dan banyak GrowthRecord; keduanya tidak dimiliki anak lain.
• Satu MilestoneResult mengacu pada banyak MilestoneQuestion melalui answers[questionId] dan versi bank soal; satu soal dapat dipakai banyak evaluasi.
• Satu Child dapat memiliki pengingat lokal untuk KPSP dan pencatatan; pengingat dibatalkan ketika anak dihapus/logout.
• Target struktur cloud: users/{uid}; users/{uid}/children/{childId}; subkoleksi milestones/{resultId} dan records/{recordId}. Nama milestones/records mengikuti AppConstants. ID berasal dari ID dokumen, tidak perlu diduplikasi sebagai field. Penghapusan dokumen induk Firestore tidak otomatis menghapus subkoleksi/foto: pembersihan wajib eksplisit oleh admin MVP.

Bagian 5 · Kontrak API

Tulis kontrak untuk setiap endpoint yang akan dipakai aplikasi. Kontrak dianggap cukup bila anggota lain bisa mengerjakannya tanpa bertanya lagi kepada Anda. Salin blok di bawah sebanyak jumlah endpoint Anda.

KEPUTUSAN KONTRAK: aplikasi menggunakan Firebase SDK, bukan server REST /api/v1. Kolom “Metode dan alamat” diisi nama operasi SDK dan path resource; “Kode status” berisi hasil Future/Stream dan kode FirebaseException, bukan HTTP 200/201 rekaan. Seluruh kontrak cloud di bawah adalah TARGET IMPLEMENTASI; koneksi Firebase, rules dan implementasi repository belum tersedia.
Konvensi umum: uid selalu dari FirebaseAuth.currentUser.uid, tidak dari input pengguna; childId/resultId/recordId adalah ID dokumen tanpa slash. u = users/{uid}; c = u/children/{childId}. Pemilik wajib cocok request.auth.uid == uid. Dokumen akun dengan deletedAt != null tidak boleh mengakses data anak (kecuali membaca/membatalkan jadwal penghapusan akun).
Jenis data cloud: tanggal pengukuran/lahir → Firestore Timestamp (tanggal lokal dikonversi konsisten); createdAt/evaluatedAt → serverTimestamp; gender/status/aspect → nama enum; id diambil dari doc.id. Reader mengubah Timestamp ke DateTime. Target nullable fields ditulis null bila kosong. Tolak field di luar daftar kontrak. GrowthRecordModel saat ini masih JSON ISO 8601, sehingga perlu adapter Firestore.
Aturan bersama: nama anak trim 2–100 karakter; nama ibu opsional ≤100; childOrder 1–9; birthDate tidak di masa depan dan usia pendaftaran maksimal 5 tahun. weightKg/heightCm dan lingkar kepala bila ada harus angka finite >0 (bukan diagnosis/rentang medis). note opsional ≤2000 karakter. recordDate antara birthDate dan hari ini. Query daftar memakai limit 20 dan cursor DocumentSnapshot terakhir; daftar kosong adalah hasil valid.
Error Firestore umum: unauthenticated → minta login ulang; permission-denied → hentikan akses, jangan coba akun lain; unavailable/deadline-exceeded → pertahankan input dan sediakan coba lagi; not-found pada operasi yang membutuhkan dokumen → tampilkan data tidak ada. get() dokumen hilang menghasilkan exists=false; list kosong bukan exception. set()/update()/delete() berhasil menghasilkan Future<void>, bukan {success:true}. Write memakai ID yang disiapkan sekali agar retry tidak menduplikasi data. Jangan menandai tersimpan di server sebelum acknowledgment; snapshot cache diberi label lokal/belum sinkron.
Bank soal dibaca lokal lewat KpspQuestionBank.getQuestions(ageMonth), tanpa endpoint, login maupun jaringan. Notifikasi juga lokal, memerlukan izin sistem; penolakan izin tidak boleh menghalangi pencatatan. todosCollection saat ini tidak dipakai, sehingga tidak menambah entitas/endpoint baru.

Endpoint 1 — Registrasi akun (SDK / target)

Metode dan alamat	FirebaseAuth.instance.createUserWithEmailAndPassword(email, password)
Kegunaan	Membuat identitas ibu. Setelah Auth sukses, lanjut operasi 2 untuk profil; pendaftaran belum lengkap jika profil belum tersimpan.
Parameter	Tidak ada path parameter. Email trim valid; password 8–128 karakter sesuai kebijakan proyek (jangan di-trim otomatis).
Data yang dikirim	{email: String, password: String}
Data yang diterima	UserCredential; user.uid dan user.email. Tidak mengharapkan JSON token dari metode ini. Simpan uid untuk menulis profil.
Kode status	Sukses: UserCredential. FirebaseAuthException: email-already-in-use, invalid-email, weak-password, operation-not-allowed, too-many-requests, network-request-failed.
Cara login	Belum perlu sesi. Firebase Auth SDK mengelola sesi/refresh token. Password hanya dikirim ke Firebase Auth melalui TLS; tidak disimpan/log. Tidak menyimpan ID token manual di Firestore atau SharedPreferences.

Endpoint 2 — Inisialisasi profil ibu (SDK / target)

Metode dan alamat	Firestore doc(u).set(profile), hanya jika dokumen belum ada
Kegunaan	Membuat profil aplikasi setelah registrasi. Jika gagal jaringan, tetap tampilkan proses lengkapi profil dan ulangi dengan uid yang sama; jangan membuat akun Auth kedua.
Parameter	uid dari sesi baru. Transaksi create-if-absent: bila dokumen sudah ada, baca saja tanpa menimpa deletedAt/createdAt.
Data yang dikirim	{email: currentUser.email, name: String|null, createdAt: serverTimestamp(), deletedAt: null, purgeAfter: null}

Data yang diterima	Future<void>; selanjutnya doc(u).get() untuk membaca timestamp server dan memetakan UserEntity.
Kode status	Sukses: Future<void>. Error: unauthenticated, permission-denied, unavailable, deadline-exceeded; validasi lokal gagal → tidak mengirim request. Lihat aturan bersama.
Cara login	Login Firebase wajib; SDK membawa kredensial otomatis. Firestore/Storage Rules memeriksa uid pemilik, akun aktif, dan keberadaan anak. Pembatasan UI saja tidak cukup. Create hanya untuk request.auth.uid == uid, email cocok token, deletedAt/purgeAfter null.

Endpoint 3 — Masuk akun (SDK / target)

Metode dan alamat	FirebaseAuth.instance.signInWithEmailAndPassword(email, password)
Kegunaan	Masuk lalu baca profil operasi 4. Akun belum lengkap menuju lengkapi profil; akun dijadwalkan hapus menuju halaman pemulihan, bukan dashboard.
Parameter	Tidak ada path parameter. Email valid; password tidak kosong.
Data yang dikirim	{email: String, password: String}
Data yang diterima	UserCredential dan currentUser. authStateChanges()/idTokenChanges() dipakai untuk navigasi dan perubahan sesi.
Kode status	Sukses: UserCredential. Error: invalid-credential (pesan umum email/kata sandi tidak cocok), invalid-email, user-disabled, too-many-requests, network-request-failed. Tangani wrong-password/user-not-found pada konfigurasi lama tanpa membocorkan keberadaan email.
Cara login	Tidak perlu sesi awal. Firebase Auth SDK mengelola sesi/refresh token. Password hanya dikirim ke Firebase Auth melalui TLS; tidak disimpan/log. Tidak menyimpan ID token manual di Firestore atau SharedPreferences.

Endpoint 4 — Baca profil akun (SDK / target)

Metode dan alamat	Firestore doc(u).get()
Kegunaan	Mengetahui profil ibu serta status aktif/masa penghapusan. Boleh dibaca pemilik saat soft-delete untuk pembatalan.
Parameter	uid dari sesi.
Data yang dikirim	Tidak ada body.
Data yang diterima	DocumentSnapshot → UserEntity ditambah deletedAt/purgeAfter pada adapter target. exists=false → lanjut lengkapi profil, bukan dianggap sukses kosong.
Kode status	Sukses: DocumentSnapshot/QuerySnapshot (boleh kosong). Error: unauthenticated, permission-denied, unavailable. Jangan tampilkan cache milik sesi sebelumnya.
Cara login	Login wajib dan request.auth.uid == uid; khusus profil akun sendiri boleh dibaca meski deletedAt terisi.

Endpoint 5 — Keluar akun (SDK / target)

Metode dan alamat	FirebaseAuth.instance.signOut()
Kegunaan	Mengakhiri sesi lokal. Hentikan listener, batalkan pengingat akun, kosongkan provider anak/jawaban/catatan dan metadata secure storage, kembali ke login.
Parameter	Tidak ada.
Data yang dikirim	Tidak ada body.
Data yang diterima	Future<void>; authStateChanges() mengirim null.

Kode status	Sukses: Future<void>. Bila platform melempar FirebaseAuthException, tampilkan gagal keluar dan jangan mengklaim sesi selesai.
Cara login	Firebase Auth SDK mengelola sesi/refresh token. Password hanya dikirim ke Firebase Auth melalui TLS; tidak disimpan/log. Tidak menyimpan ID token manual di Firestore atau SharedPreferences.

Endpoint 6 — Tambah profil anak (SDK / target)

Metode dan alamat	Firestore collection(u + /children).doc(childId).set(child)
Kegunaan	Menyimpan profil anak. childId dibuat dengan collection.doc().id sekali sebelum percobaan pertama.
Parameter	uid; childId baru; validasi nama, usia, gender dan childOrder mengikuti aturan bersama.
Data yang dikirim	{parentId: uid, name: String, birthDate: Timestamp, gender: "male"|"female", childOrder: int, photoUrl: null, createdAt: serverTimestamp()}
Data yang diterima	Future<void>; childId sudah diketahui dari DocumentReference. Reader memakai doc.id.
Kode status	Sukses: Future<void>. Error: unauthenticated, permission-denied, unavailable, deadline-exceeded; validasi lokal gagal → tidak mengirim request. Lihat aturan bersama.
Cara login	Login Firebase wajib; SDK membawa kredensial otomatis. Firestore/Storage Rules memeriksa uid pemilik, akun aktif, dan keberadaan anak. Pembatasan UI saja tidak cukup. Rules create memeriksa parentId == uid, tipe/nilai data, dan createdAt == request.time.

Endpoint 7 — Daftar profil anak (SDK / target)

Metode dan alamat	Firestore collection(u + /children).orderBy(createdAt, descending: true).limit(20).get()/snapshots()
Kegunaan	Menampilkan anak milik ibu pada dashboard dan memasok pilihan evaluasi/catatan.
Parameter	uid; cursor opsional startAfterDocument(lastDoc). Urutan createdAt menurun, ID dokumen sebagai pengikat urutan bawaan.
Data yang dikirim	Tidak ada body; cursor dari halaman sebelumnya.
Data yang diterima	QuerySnapshot → List<ChildEntity>; [] jika belum ada anak. Halaman berikutnya memakai doc terakhir; timestamp lokal pending ditandai belum sinkron.
Kode status	Sukses: DocumentSnapshot/QuerySnapshot (boleh kosong). Error: unauthenticated, permission-denied, unavailable. Jangan tampilkan cache milik sesi sebelumnya.
Cara login	Login Firebase wajib; SDK membawa kredensial otomatis. Firestore/Storage Rules memeriksa uid pemilik, akun aktif, dan keberadaan anak. Pembatasan UI saja tidak cukup.

Endpoint 8 — Ubah profil anak (SDK / target)

Metode dan alamat	Firestore doc(c).update(patch)
Kegunaan	Target edit nama/tanggal lahir/gender/urutan anak. Tidak mengubah kepemilikan atau mengubah hasil evaluasi historis otomatis.
Parameter	uid; childId yang ada. Tolak patch kosong dan field di luar whitelist.
Data yang dikirim	Patch subset {name, birthDate, gender, childOrder}; validasi sama seperti tambah. parentId, id dan createdAt tidak boleh diubah.

Data yang diterima	Future<void>; stream dashboard memperbarui profil. Hitung ulang usia untuk evaluasi berikutnya.
Kode status	Sukses: Future<void>. Error: unauthenticated, permission-denied, unavailable, deadline-exceeded; validasi lokal gagal → tidak mengirim request. Lihat aturan bersama. not-found bila childId tidak ada.
Cara login	Login Firebase wajib; SDK membawa kredensial otomatis. Firestore/Storage Rules memeriksa uid pemilik, akun aktif, dan keberadaan anak. Pembatasan UI saja tidak cukup.

Endpoint 9 — Hapus profil anak (SDK / target)

Metode dan alamat	Firestore doc(c).delete(), setelah pembersihan data turunan terkonfirmasi
Kegunaan	Target hapus profil dengan konfirmasi nama anak. MVP: pembersihan subkoleksi milestones/records dan foto dilakukan admin secara eksplisit; app tidak boleh mengklaim seluruh data terhapus hanya karena dokumen induk hilang.
Parameter	uid; childId; pembatalan notifikasi lokal anak. Operasi permanen tidak memiliki undo setelah admin membersihkan data.
Data yang dikirim	Tidak ada body. Admin membersihkan descendants dan Storage lebih dahulu; lalu hapus child. Tidak mengandalkan cascade Firestore.
Data yang diterima	Future<void>; dokumen tidak ada diperlakukan selesai/idempoten. Dashboard menghapus anak setelah acknowledgment.
Kode status	Sukses: Future<void>. Error: unauthenticated, permission-denied, unavailable, deadline-exceeded; validasi lokal gagal → tidak mengirim request. Lihat aturan bersama.
Cara login	Pemilik mengajukan/mengonfirmasi; eksekusi cleanup MVP oleh admin tepercaya. Tidak menaruh service-account key dalam aplikasi. Sebelum mekanisme cleanup siap, fitur hapus total belum boleh dinyatakan selesai.

Endpoint 10 — Simpan hasil KPSP (SDK / target)

Metode dan alamat	Firestore collection(c + /milestones).doc(resultId).set(result)
Kegunaan	Mencatat skrining dan riwayat sesuai bank soal tervalidasi. Buat resultId sekali; dokumen immutable setelah tersimpan.
Parameter	uid; childId; resultId; ageMonthEvaluated anggota daftar usia; questionBankVersion versi paket soal yang digunakan.
Data yang dikirim	{childId, ageMonthEvaluated: int, answers: Map<String,bool>, totalYes: int, status: "sesuai"|"meragukan"|"penyimpangan", evaluatedAt: serverTimestamp(), questionBankVersion: String}. Semua ID jawaban tepat sama dengan soal pada usia/versi tersebut; hitung ulang total dan status, jangan percaya angka kiriman UI.
Data yang diterima	Future<void>; hasil lokal masih ditandai belum sinkron sebelum acknowledgment. Foto tidak dikirim dalam dokumen hasil.
Kode status	Sukses: Future<void>. Error: unauthenticated, permission-denied, unavailable, deadline-exceeded; validasi lokal gagal → tidak mengirim request. Lihat aturan bersama. Hasil invalid/incomplete tidak boleh disimpan; rules harus membatasi usia, kunci jawaban, total dan status sesuai bank yang dirilis.
Cara login	Login Firebase wajib; SDK membawa kredensial otomatis. Firestore/Storage Rules memeriksa uid pemilik, akun aktif, dan keberadaan anak. Pembatasan UI saja tidak cukup. Status merupakan hasil skrining, bukan diagnosis. Rules validasi harus selaras bank/versi sebelum release; bukan mengklaim Firebase otomatis memahami KPSP.

Endpoint 11 — Baca riwayat KPSP (SDK / target)

Metode dan alamat	Firestore collection(c + /milestones).orderBy(evaluatedAt, descending: true).limit(20).get()/snapshots()
Kegunaan	Menampilkan hasil terbaru dan riwayat anak yang dipilih, tanpa menimpa jawaban evaluasi berjalan.
Parameter	uid; childId; cursor opsional startAfterDocument(lastDoc).
Data yang dikirim	Tidak ada body.
Data yang diterima	QuerySnapshot → List<MilestoneResultEntity>; adapter juga mempertahankan questionBankVersion. [] jika belum ada evaluasi.
Kode status	Sukses: DocumentSnapshot/QuerySnapshot (boleh kosong). Error: unauthenticated, permission-denied, unavailable. Jangan tampilkan cache milik sesi sebelumnya.
Cara login	Login Firebase wajib; SDK membawa kredensial otomatis. Firestore/Storage Rules memeriksa uid pemilik, akun aktif, dan keberadaan anak. Pembatasan UI saja tidak cukup.

Endpoint 12 — Simpan catatan pertumbuhan (SDK / target)

Metode dan alamat	Firestore collection(c + /records).doc(recordId).set(record)
Kegunaan	Menyimpan pengukuran/catatan. recordId dibuat sekali; dokumen tidak dikirim ulang dengan ID baru saat retry.
Parameter	uid; childId; recordId; validasi angka, tanggal dan panjang catatan seperti aturan bersama.
Data yang dikirim	{childId, recordDate: Timestamp, weightKg: number, heightCm: number, headCircumferenceCm: number|null, note: String|null, photoUrl: String|null, createdAt: serverTimestamp()}. photoUrl target berisi path Storage privat dari operasi 15, bukan file biner atau URL publik.
Data yang diterima	Future<void>; sukses server → pesan berhasil dan kembali ke home. Gagal → input tetap ada; foto orphan dibersihkan saat retry/admin.
Kode status	Sukses: Future<void>. Error: unauthenticated, permission-denied, unavailable, deadline-exceeded; validasi lokal gagal → tidak mengirim request. Lihat aturan bersama.
Cara login	Login Firebase wajib; SDK membawa kredensial otomatis. Firestore/Storage Rules memeriksa uid pemilik, akun aktif, dan keberadaan anak. Pembatasan UI saja tidak cukup.

Endpoint 13 — Baca catatan pertumbuhan (SDK / target)

Metode dan alamat	Firestore collection(c + /records).orderBy(recordDate, descending: true).limit(20).get()/snapshots()
Kegunaan	Menampilkan catatan terbaru dan riwayat berat/tinggi anak.
Parameter	uid; childId; cursor opsional startAfterDocument(lastDoc).
Data yang dikirim	Tidak ada body.
Data yang diterima	QuerySnapshot → List<GrowthRecordEntity>. createdAt dibaca sebagai DateTime dari Timestamp, bukan DateTime.parse(Timestamp). [] jika belum ada catatan.
Kode status	Sukses: DocumentSnapshot/QuerySnapshot (boleh kosong). Error: unauthenticated, permission-denied, unavailable. Jangan tampilkan cache milik sesi sebelumnya.

Cara login	Login Firebase wajib; SDK membawa kredensial otomatis. Firestore/Storage Rules memeriksa uid pemilik, akun aktif, dan keberadaan anak. Pembatasan UI saja tidak cukup.

Endpoint 14 — Hapus catatan pertumbuhan (SDK / target)

Metode dan alamat	Firestore doc(c + /records/{recordId}).delete(); foto terkait memakai operasi 17
Kegunaan	Menghapus catatan setelah konfirmasi. Saat gagal jangan hilangkan card secara permanen. Jika ada foto, hapus objek terkait lebih dahulu; bila objek sudah hilang, lanjut hapus record.
Parameter	uid; childId; recordId. Baca path foto dari record milik sendiri; jangan menerima URL/path arbitrer dari pengguna.
Data yang dikirim	Tidak ada body.
Data yang diterima	Future<void>; setelah server sukses perbarui daftar. Catatan tidak ada → selesai/idempoten.
Kode status	Sukses: Future<void>. Error: unauthenticated, permission-denied, unavailable, deadline-exceeded; validasi lokal gagal → tidak mengirim request. Lihat aturan bersama.
Cara login	Login Firebase wajib; SDK membawa kredensial otomatis. Firestore/Storage Rules memeriksa uid pemilik, akun aktif, dan keberadaan anak. Pembatasan UI saja tidak cukup.

Endpoint 15 — Unggah foto catatan (SDK / target)

Metode dan alamat	FirebaseStorage.instance.ref(path).putData(bytes, SettableMetadata(contentType: mime))
Kegunaan	Target foto opsional JPEG/PNG maksimal 2 MiB (2.097.152 byte); tanpa video. Tidak ada endpoint foto dalam aplikasi saat ini.
Parameter	path = users/{uid}/children/{childId}/records/{recordId}/photo.jpg atau photo.png; recordId dibuat sebelum upload. Satu foto per record.
Data yang dikirim	Bytes nonkosong ≤2.097.152; mime image/jpeg atau image/png. Klien harus decode dan encode ulang gambar untuk validasi serta membuang metadata EXIF/lokasi, bukan hanya percaya ekstensi.
Data yang diterima	UploadTask → TaskSnapshot; simpan ref.fullPath sebagai photoUrl (konvensi target, nama field lama dipertahankan). Setelah upload berhasil baru simpan record. Gagal record write → ulangi memakai ID/path sama atau hapus orphan.
Kode status	Sukses: TaskSnapshot.state success. FirebaseException Storage: unauthenticated, unauthorized, retry-limit-exceeded, canceled, quota-exceeded, unknown; ukuran/format salah ditolak lokal dan Rules.
Cara login	Login Firebase wajib; SDK membawa kredensial otomatis. Firestore/Storage Rules memeriksa uid pemilik, akun aktif, dan keberadaan anak. Pembatasan UI saja tidak cukup. Storage Rules: auth.uid==uid, akun aktif, anak ada, ukuran ≤2.097.152 dan contentType tepat JPEG/PNG. Metadata MIME saja bukan bukti isi aman.

Endpoint 16 — Baca foto privat (SDK / target)

Metode dan alamat	FirebaseStorage.instance.ref(photoUrl).getData(2097152)
Kegunaan	Menampilkan foto hanya kepada pemilik lewat byte download terautentikasi. Tidak membagikan tokenized getDownloadURL untuk data anak.
Parameter	photoUrl harus prefix path anak/record pemilik; bukan URL eksternal bebas.
Data yang dikirim	Tidak ada body.

Data yang diterima	Future<Uint8List?> untuk Image.memory; null/objek hilang → placeholder, tidak crash.
Kode status	Sukses: bytes. Error: object-not-found, unauthenticated, unauthorized, retry-limit-exceeded. Foto tidak boleh masuk log/analytics.
Cara login	Login Firebase wajib; SDK membawa kredensial otomatis. Firestore/Storage Rules memeriksa uid pemilik, akun aktif, dan keberadaan anak. Pembatasan UI saja tidak cukup.

Endpoint 17 — Hapus foto privat (SDK / target)

Metode dan alamat	FirebaseStorage.instance.ref(photoUrl).delete()
Kegunaan	Menghapus foto saat record dihapus atau upload orphan dibatalkan.
Parameter	Path tepat milik uid/childId/recordId. Tidak melakukan penghapusan folder/prefix milik orang lain.
Data yang dikirim	Tidak ada body.
Data yang diterima	Future<void>; object-not-found dianggap sudah terhapus.
Kode status	Sukses: Future<void>. Error: unauthorized, unauthenticated, retry-limit-exceeded; selain object-not-found harus dilaporkan dan dapat dicoba lagi.
Cara login	Login Firebase wajib; SDK membawa kredensial otomatis. Firestore/Storage Rules memeriksa uid pemilik, akun aktif, dan keberadaan anak. Pembatasan UI saja tidak cukup.

Endpoint 18 — Jadwalkan penghapusan akun 30 hari (SDK / target)

Metode dan alamat	Firestore runTransaction: baca doc(u), lalu update deletedAt dan purgeAfter
Kegunaan	Target soft-delete: akun tidak bisa membuka data anak selama menunggu penghapusan. Tidak menjalankan FirebaseAuth.currentUser.delete() di awal, agar pembatalan masih mungkin.
Parameter	uid; konfirmasi ulang password melalui reauthenticateWithCredential(EmailAuthProvider.credential(...)). Waktu dari snapshot created server marker/aturan request.time; jangan percaya jam perangkat untuk otorisasi.
Data yang dikirim	{deletedAt: Timestamp, purgeAfter: Timestamp}. Aturan server wajib memastikan deletedAt sesuai request.time dan purgeAfter = deletedAt + duration.value(30,"d"). Implementasi penentuan Timestamp harus diuji terhadap rules; field baru ini belum dibuat.
Data yang diterima	Future<void>; tampilkan tanggal purgeAfter dari snapshot server. Batalkan reminder, kosongkan state sensitif dan logout. Admin MVP membersihkan Auth/Firestore/Storage setelah jatuh tempo, bukan janji penghapusan otomatis.
Kode status	Sukses: transaksi committed. Error: aborted, permission-denied, unavailable; Auth: invalid-credential, requires-recent-login. Jangan keluar sebelum request penjadwalan terkonfirmasi.
Cara login	Pemilik + recent authentication; rules memeriksa auth_time, transisi null→terjadwal, dan tidak mengizinkan perubahan profil lain sekaligus.

Endpoint 19 — Batalkan penghapusan akun (SDK / target)

Metode dan alamat	Firestore runTransaction: baca doc(u), lalu update({deletedAt:null, purgeAfter:null})
Kegunaan	Memulihkan akun dalam masa 30 hari. Login hanya membuka halaman status/pemulihan sampai pembatalan berhasil.
Parameter	uid; dokumen masih ada, deletedAt != null, request.time < purgeAfter. Setelah jatuh tempo tidak ada jaminan pemulihan.

Data yang dikirim	{deletedAt: null, purgeAfter: null}; hanya dua field ini dapat diubah pada operasi pembatalan.
Data yang diterima	Future<void>; baca ulang profil, muat ulang data anak dan tawarkan mengaktifkan pengingat kembali.
Kode status	Sukses: transaksi committed. Error: permission-denied bila lewat batas/bukan pemilik, not-found bila sudah dibersihkan, aborted, unavailable.
Cara login	Pemilik akun; aturan pengecualian pembatalan tidak boleh sekaligus membuka akses anak untuk akun lain/akun yang masih terjadwal hapus.

Bagian 6 · Risiko Keamanan dan Cara Mengatasinya

Isi minimal empat risiko. Mitigasi harus konkret — "menggunakan login" belum cukup, sebutkan caranya.

RISIKO	JENIS	CARA MENGATASINYA
Data anak terbaca/diubah akun lain (IDOR)	Otorisasi / privasi	TARGET: Rules default-deny; request.auth != null dan request.auth.uid == uid pada path users/{uid}/children/...; parentId saat create harus uid dan immutable. Validasi existence akun/anak dan status deletedAt. Uji emulator akun A ditolak membaca/menulis path akun B. Rules belum tersedia; /home masih bypass login untuk pengujian.
Password bocor / percobaan login berulang	Autentikasi	TARGET: Firebase Auth email-password melalui TLS; password/token tidak masuk Firestore, log atau pesan error. Gunakan pesan invalid-credential yang sama, tangani too-many-requests, dan kebijakan password. SDK mengelola refresh sesi; router mendengar authStateChanges dan token errors. Login belum terimplementasi.
Unggah foto salah format, besar, atau mengandung lokasi anak	Validasi file / privasi	TARGET: decode/encode ulang JPEG/PNG dan buang EXIF; bytes ≤2.097.152. Storage Rules membatasi path pemilik, size dan contentType. Untuk delete gunakan request.resource==null branch, bukan membaca size dari null. Baca dengan getData terautentikasi; jangan bagikan download URL ber-token. Picker/upload belum tersedia.
Input tidak valid atau skor KPSP dimanipulasi	Integritas / keselamatan	SAAT INI: validasi form masih terbatas; state jawaban global dan bank soal perlu validasi. TARGET: validasi tipe/rentang teknis di client serta rules, semua ID jawaban wajib cocok bank/versi, hitung ulang total/status. Validasi formulir dan gambar dengan sumber otoritatif sebelum release; tampilkan “skrining, bukan diagnosis” dan arahan konsultasi. Jangan menganggap test jumlah soal membuktikan validitas medis.
Sisa data di perangkat setelah logout	Kerahasiaan lokal	TARGET: hentikan listener; reset provider anak, jawaban dan catatan; hapus user_id/user_email dari secure storage; batalkan reminder. Untuk MVP data sensitif, nonaktifkan persistent cache Firestore sebelum query dan gunakan memori; jangan mengaktifkan web persistence tanpa kebijakan perangkat bersama. SystemUiOverlayStyle hanya mengatur status bar, bukan mencegah screenshot. SecureStorageService belum berarti semua data anak terenkripsi.
Kunci admin/signing bocor ke repositori	Konfigurasi / kredensial	TARGET: jangan commit service-account private key, signing keystore/password atau token. Rotasi bila bocor; batasi API keys sesuai aplikasi/API; deploy rules dan pertimbangkan App Check saat produksi. google-services.json adalah konfigurasi klien, bukan private key/admin credential; meng-ignore file tersebut saja tidak mengamankan database. Di proyek file itu sudah di-ignore; simpan panduan konfigurasi terpisah untuk tim.

RISIKO	JENIS	CARA MENGATASINYA
Penghapusan menghilangkan induk tetapi meninggalkan foto/subkoleksi	Siklus hidup data	TARGET: soft-delete 30 hari membatasi akses langsung saat terjadwal; pembatalan hanya sebelum purgeAfter. Admin MVP wajib memeriksa jatuh tempo dan membersihkan descendants, Storage, user document serta Firebase Auth. Catat keberhasilan/gagal cleanup tanpa menyalin data sensitif. Penghapusan induk bukan cascade; jangan menjanjikan penghapusan otomatis tepat waktu sebelum job server dibuat.

Daftar Periksa Sebelum Mengumpulkan

- Diagram arsitektur lengkap dan setiap bagiannya terhubung jelas.
- Kontrak API cukup rinci untuk dikerjakan tanpa bertanya lagi.
- Setiap entitas pada model data terpakai minimal satu user story.
- Setiap risiko keamanan punya cara mengatasi yang spesifik.
- Dokumen diunggah ke LMS dan salinannya ada di folder /docs repositori kelompok.

Status pengumpulan: file Word/PDF disimpan di app/docs (root /docs saat folder app dijadikan root repositori). Belum diunggah ke LMS dan belum di-push ke GitHub. Identitas anggota/peran dan rujukan user story Milestone 1 belum terkonfirmasi.

Batas serah Sesi 5, pukul 23.59. Satu berkas PDF per kelompok, diunggah Project Manager. Bobot 7%, dinilai dengan Rubrik Milestone 2 pada Panduan Final Project.